

Entity Component System

Inhalt

- Was ist ein ECS?
 - Entities
 - Components
 - Systems
- Wie Funktioniert ein ECS?
 - Registry/World
 - Component Pool

Inhalt

- Warum?
 - Vererbung
 - Vorteile
 - Performance

Was ist ein ECS?

- Software-Architekturmuster
- Spielwelt-Objektdarstellung

	Entity 1	Entity 2	Entity 3
Component 1	[Data]		
Component 2	[Data]	[Data]	[Data]
Component 3		[Data]	[Data]

Entities

- Sind die Spiel-Objekte
- Ist nur ein Integer als ID

```
7      using Entity = std::uint32_t;
```

Components

- Speichern die Informationen einer Entity
- Sind Strukturen
- Beispiele: Position, Health

```
4      struct TestComponent {  
5          const char* name = "TestComponent";  
6          int value;  
7          bool active;  
8      };
```

Systems

- Agieren über gewünschte Komponenten Pools
- Sind einfache Funktionen
- Speichert keine Daten

```
4 struct Position {
5     int x;
6     int y;
7 };
8
9 struct Velocity {
10    int dx;
11    int dy;
12 };
13
16 void mainloop(ecs::Registry& registry) {
17     registry.view<Position, Velocity>().each([](ecs::Entity entity, Position& pos, Velocity& vel) {
18         MovementSystem(entity, pos, vel);
19     });
20 }
21
22 void MovementSystem(ecs::Entity entity, Position& pos, Velocity& vel) {
23     pos.x += vel.dx;
24     pos.y += vel.dy;
25 }
```


Wie Funktioniert ein ECS?

- Registry/World
- Komponenten Pools

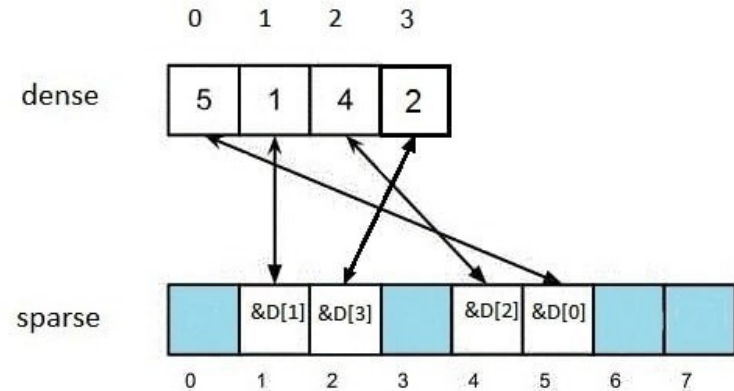
Registry/World

- Core System von einem ECS
- Speichert alle Komponenten Pools
- Verwaltet Entity-IDs
- Stellt Komponenten Systemen zur Verfügung

```
12     class Registry {
13     public:
14         Registry() = default;
15         ~Registry() = default;
16
17         Entity createEntity();
18
19         template<typename T>
20         void add(Entity entity, T component);
21
22         template<typename T>
23         void rm(Entity entity);
24
25         template<typename T>
26         T* get(Entity entity);
27
28         template<typename T>
29         bool has(Entity entity);
30
31         template<typename... Components>
32         View<Components...> view();
33
34     private:
35         template<typename T>
36         SparseSet<T>& getOrCreateSparseSet();
37
38         Entity nextEntityID = 0;
39         std::unordered_map<std::type_index, std::unique_ptr<ISparseSet>> componentSets;
40     }; // class: Registry
```

Component Pool

- Speichert alle Instanzen eines Komponententyps in Arrays
- Zugriff, Hinzufügen, Löschen haben eine konstante Zeit ($O(1)$)



```
10     class ISparseSet {
11     public:
12         virtual ~ISparseSet() = default;
13         virtual void remove(Entity entity) = 0;
14     };
15
16     template <typename T>
17     class SparseSet : public ISparseSet {
18     public:
19         void add(Entity entity, T component);
20         void remove(Entity entity);
21         T* get(Entity entity);
22         bool contains(Entity entity);
23
24         auto begin() { return dense.begin(); }
25         auto end() { return dense.end(); }
26         auto cbegin() { return dense.cbegin(); }
27         auto cend() { return dense.cend(); }
28
29         const std::vector<Entity>& getEntities() const { return indices; }
30         size_t size() const { return dense.size(); }
31
32     private:
33         std::vector<Entity> sparse;
34         std::vector<Entity> indices;
35         std::vector<T> dense;
36     }; // class: SparseSet
```

Std Library

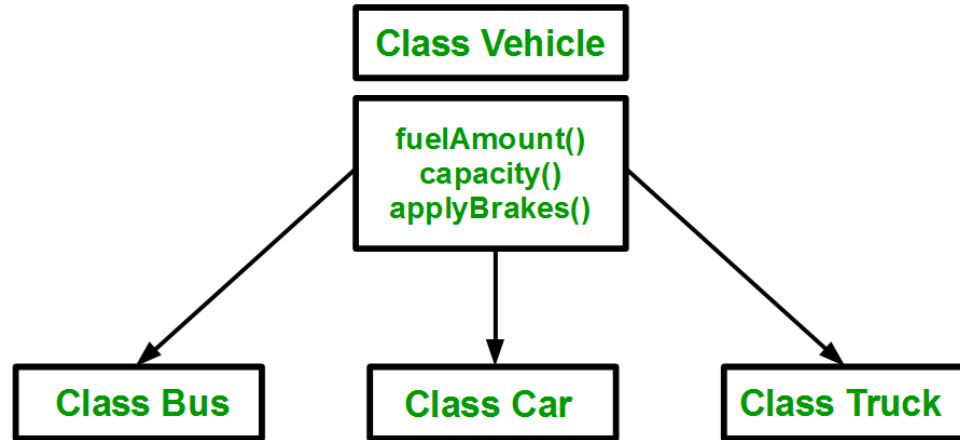
- Genutzt
 - unordered_map (Komponenten-Pool speicherung)
 - Typeindex (Komponenten-Pool speicherung)
 - Memory (Komponenten-Pool speicherung)
 - Vector (Komponentenspeicherung)

Warum?

- Warum eine komplizierte Registry?
- Warum ein Komponenten Pool mit drei verschiedenen Arrays?

Vererbung

- Klassische OOP
- Einfacherer Struktur (hierarchisches design)



Das Problem

- Starre Hierarchien
- Schwer in der Laufzeit zu ändern
- Diamant Problem

Vorteile von ECS

- Nur eine Liste von Komponenten
- Änderungen an Entities während der Laufzeit möglich

Performance

- Komponenten liegen im Ram direkt nebeneinander
- Höhere chance auf cache-hits